SISTEMAS OPERATIVOS

Aula Teórica 2

Daniel Castro

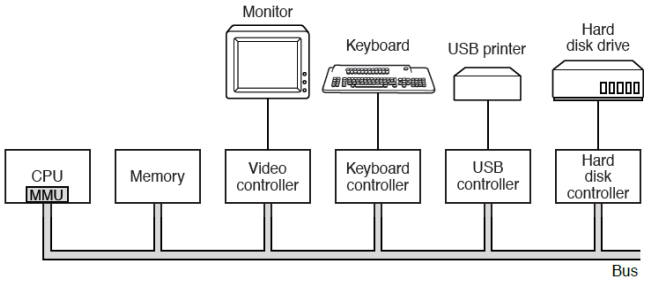
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

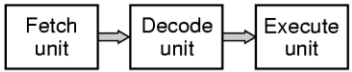
The Operating System Zoo

- Mainframe operating systems
- Server operating systems
- Personal computer operating systems
- Smartphone and Handheld Computer operating systems
- The IoT and Embedded operating systems
- Real-time operating Systems
- Smart Card operating systems

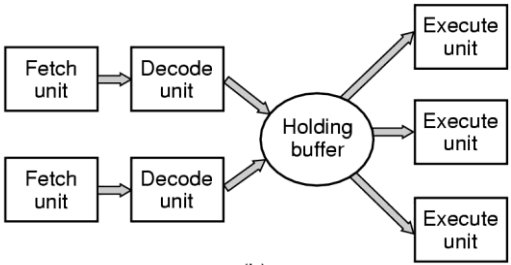
Computer Hardware Review (Processors)



- Components of a simple personal computer



(a)



(b)

- (a) A three-stage pipeline
- (b) A superscalar CPU

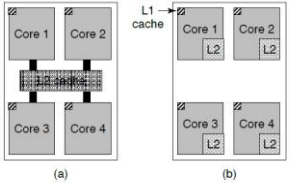


Figure 1-8. (a) A quad-core chip with a shared L2 cache. (b) A quad-core chip with separate L2 caches.

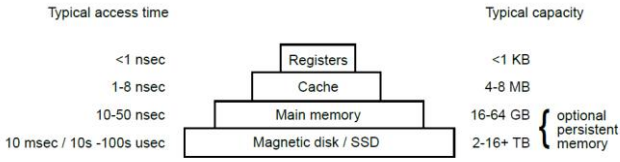
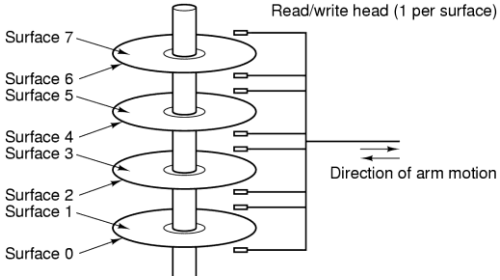


Figure 1-9. A typical memory hierarchy. The numbers are very rough approximations.

- Typical memory hierarchy
  - numbers shown are rough approximations



- Structure of a disk drive

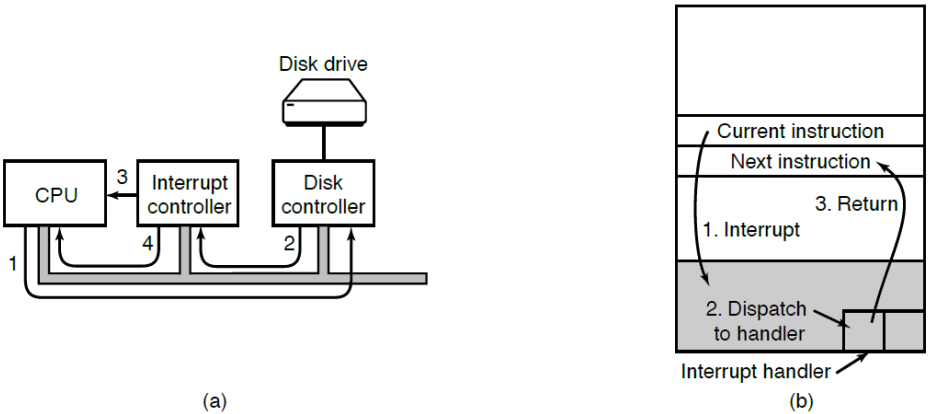
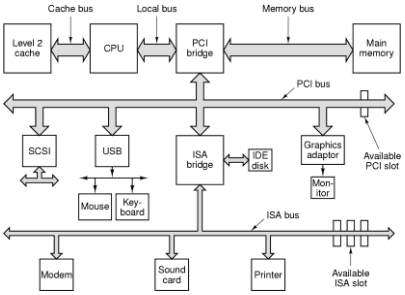


Figure 1-11. (a) The steps in starting an I/O device and getting an interrupt. (b) Interrupt processing involves taking the interrupt, running the interrupt handler, and returning to the user program.



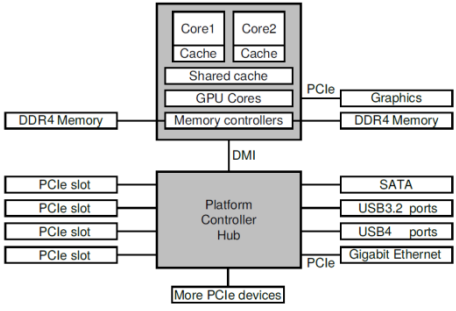
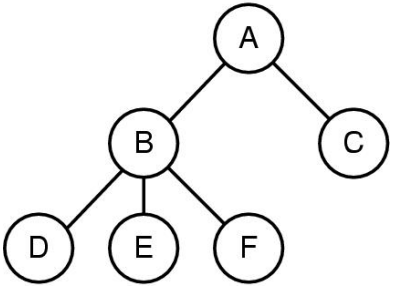
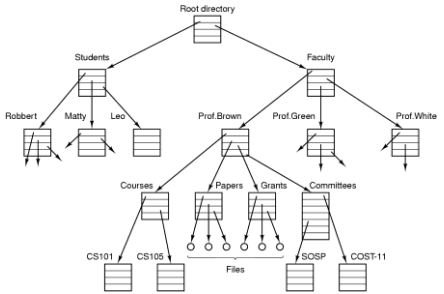


Figure 1-12. The structure of a large x86 system.

- Structure of a large Pentium system

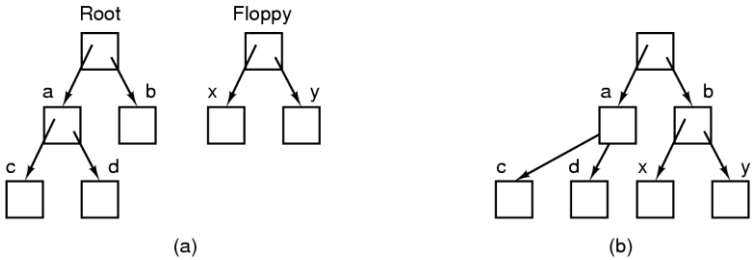


- A process tree
  - A created two child processes, B and C
  - B created three child processes, D, E, and F

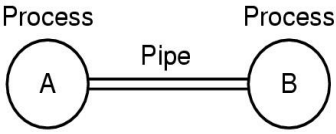


- File system for a university department

Operating System Concepts (Root file system)



- Before mounting,
  - files on floppy are inaccessible
- After mounting floppy on b,
  - files on floppy are part of file hierarchy



- Two processes connected by a Pipe.
  A Pipe is a sort of pseudofile that can be used to connect two processes,

Steps in Making a System Call

```
count = read(fd, buffer, nbytes);
```

Take a quick look at the read system call.
The first parameter specifies the file, the second one pointing to the buffer, and the third one gives the number of bytes to read.
System calls are performed in a series of steps. To make this concept clearer, let us examine the read call discussed above. In preparation for calling the *read* library procedure, which actually makes the read system call, the calling program first prepares the parameters, for instance by storing them in a set of registers that by convention are used for parameters. For instance, on x86-64 CPUs, Linux, FreeBSD, Solaris, and macOS use the System V AMD64 ABI **calling convention,** which means that the first six parameters are passed in registers RDI, RSI, RDX, RCX, R8, and R9. If there are more than six arguments, the remainder will be pushed onto the stack. As we have only three arguments for *read library procedure,* this is shown as steps 1–3 in Fig. 1-17.

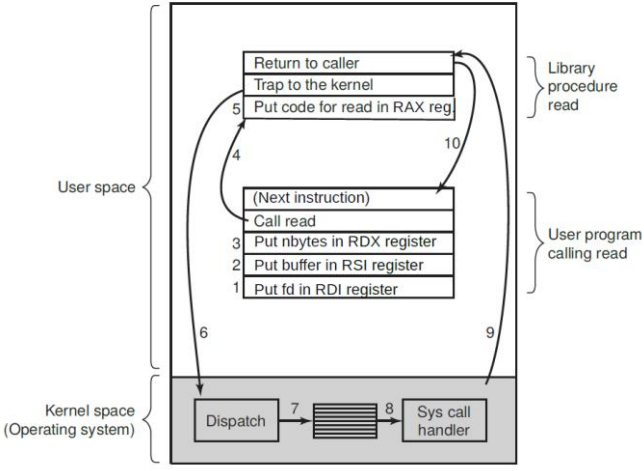


Figure 1-17. The 10 steps in making the system call read(fd, buffer, nbytes).

Some System Calls For Process Management

POSIX (International Standard 9945-1). There are about 100 procedure calls.

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

Figure 1-18. Some of the major POSIX system calls. The return code *s* is −1 if an error has occurred. The return codes are as follows: *pid* is a process id, *fd* is a file descriptor, *n* is a byte count, *position* is an offset within the file, and *seconds* is the elapsed time. The parameters are explained in the text.

Some System Calls For File Management

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Some System Calls For Directory Management

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Some System Calls For Miscellaneous Tasks

Miscellaneous

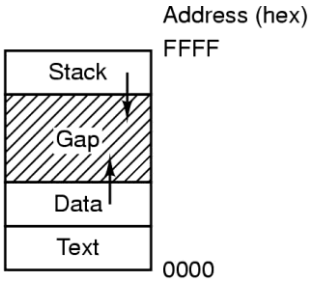
Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

System Calls (a stripped down shell)

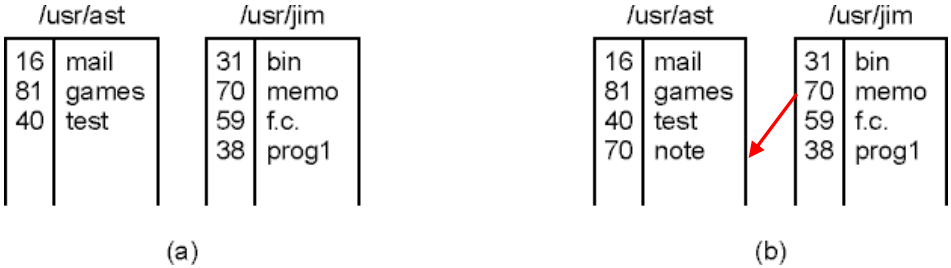
```
1  A stripped down shell:
2  while (TRUE) {                                  /* repeat forever */
3       type_prompt( );                            /* display prompt on the screen */
4       read_command (command, parameters)         /* input from terminal */
5  if (fork() != 0) {                              /* fork off child process */
6       /* Parent code */
7       waitpid( -1, &status, 0);                  /* wait for child to exit */
8  } else {
9       /* Child code */
10      execve (command, parameters, 0);           /* execute command */
11   }
12 }
```

System Calls (2)

Processes in UNIX have their memory divided up into three segments: the **text segment (i.e., the program code),** the **data segment (i.e., the variables),** and the **stack segment.**

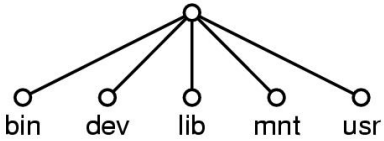


- Processes have three segments: text, data, stack

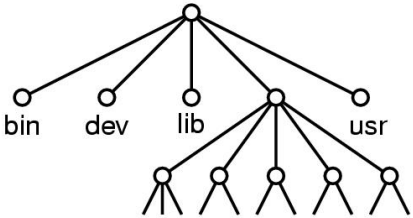


link("/usr/jim/memo", "/usr/ast/note");

- (a) Two directories before linking */usr/jim/memo* to ast's directory
- (b) The same directories after linking



(a)



(b)

```
mount("/dev/sdb0", "/mnt", 0);
```

- File system before the mount
- File system after the mount

System Calls (the Windows API)

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Figure 1-23. The Win32 API calls that roughly correspond to the UNIX calls of Fig. 1-18. It is worth emphasizing that Windows has a very large number of other system calls, most of which do not correspond to anything in UNIX.

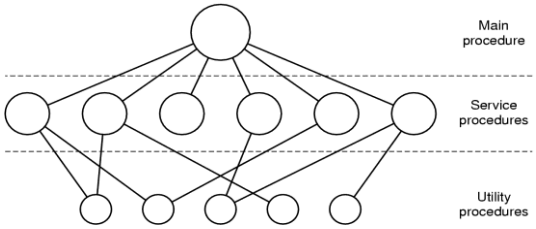
With UNIX there is almost a one-to-one relationship between the system call (e.g. read) and the library procedures (e.g. *read*) usedv to invoke the system calls.

Windows is radically different. The library calls and the actual system calls are **highly decoupled**. The number of **Win32 API (Application Programming Interface)** are extremely large, numbering in the thousands.
With Windows it is impossible to see what is a system call (i.e., performed by the kernel) and what is simply a user-space library call. In fact, what is a system call in one version of Windows may be done in user space in a different version, and vice versa.

Operating System Structure (Monolithic Systems)

This organization suggests a basic structure for the operating system:
1. A main program that invokes the requested service procedure.
2. A set of service procedures that carry out the system calls.

3. A set of utility procedures that help the service procedures.



- Everything is put below the system-call interface and above the physical hardware
- Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level
- e.g. , Unix, Linux, MS-DOS, Windows 9* family

Benefits:
FAST!

Detriments:
Less reliable (more code is running in kernel mode)
Less secure
Not easy to port to new architectures (…it depends)

Operating System Structure (Layered Systems)

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

- Structure of the THE (Dutch computer, the Electrologica X8, which had 32K of 27-bit words) operating system.

Operating System Structure (Microkernels)

The MINIX 3 microkernel is only about 15,000 lines of C and some 1400 lines of assembler for very low-level functions such as catching interrupts and switching processes.

The C code manages and schedules processes, handles interprocess communication (by passing messages between processes), and offers a set of about 40 kernel calls to allow the rest of the operating system to do its work.

The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space. Communication is provided through message passing.
e.g., Mach, XNU (used in OpenStep and Darwin, part of MAC OS X)

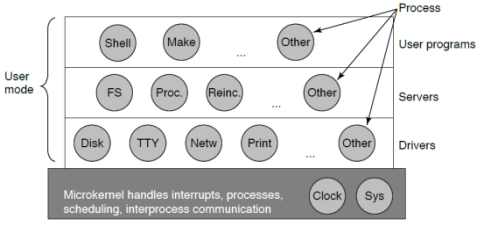
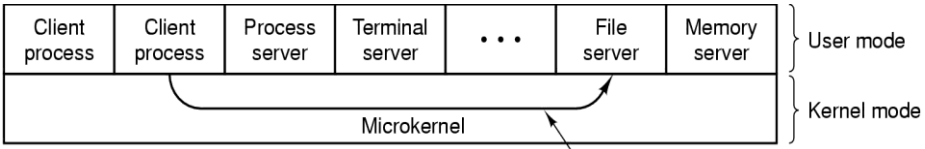


Figure 1-26. Simplified structure of the MINIX system.

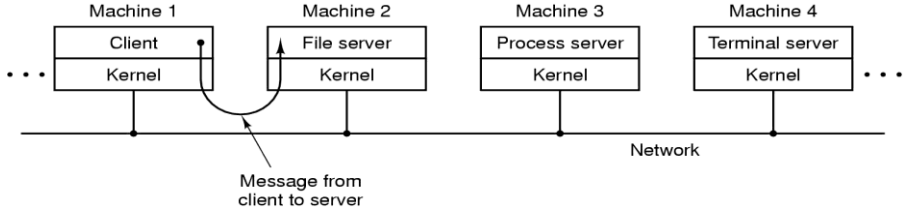
Benefits:
Easier to extend a microkernel
Easier to port the operating system to new architectures
More reliable (less code is running in kernel mode)
More secure

Detriments:
SLOW: Performance overhead of user space to kernel space communication

Operating System Structure (cliente-server model)



- The client-server model



- The client-server model in a distributed system

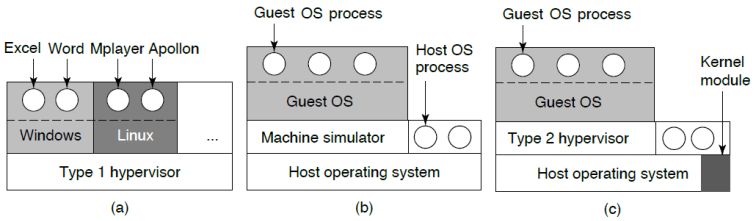


Figure 1-29. (a) A type 1 hypervisor. (b) A pure type 2 hypervisor. (c) A practical type 2 hypervisor.

Metric Units

Exp.	Explicit	Prefix	Exp.	Explicit	Prefix
10^{-3}	0.001	milli	10^{3}	1,000	Kilo
10^{-6}	0.000001	micro	10^{6}	1,000,000	Mega
10^{-9}	0.000000001	nano	10^{9}	1,000,000,000	Giga
10^{-12}	0.000000000001	pico	10^{12}	1,000,000,000,000	Tera
10^{-15}	0.000000000000001	femto	10^{15}	1,000,000,000,000,000	Peta
10^{-18}	0.000000000000000001	atto	10^{18}	1,000,000,000,000,000,000	Exa
10^{-21}	0.000000000000000000001	zepto	10^{21}	1,000,000,000,000,000,000,000	Zetta
10^{-24}	0.000000000000000000000001	yocto	10^{24}	1,000,000,000,000,000,000,000,000	Yotta

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

